PPO ON GRID2OP

daniele paletti - repository

OVERVIEW

- Baseline training
- Baseline improvement
- Multi-Agent PPO

BASELINE TRAINING

- Model description
- Environment
- Training methodology
- Trained agent analysis

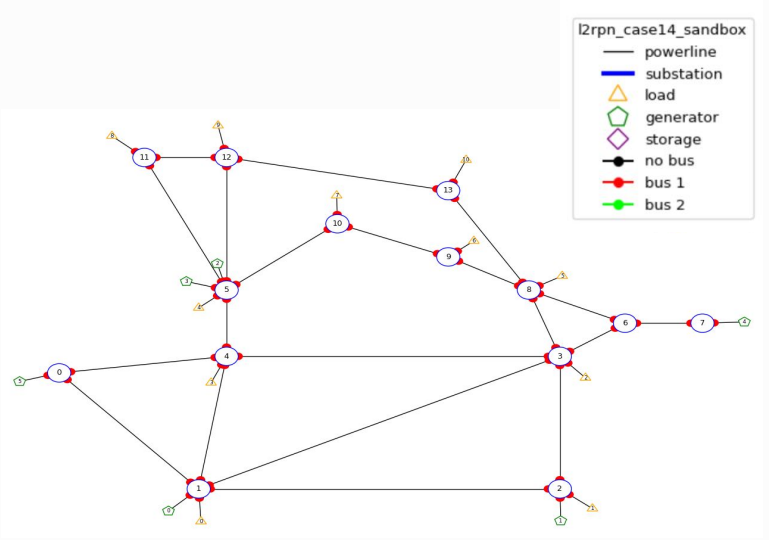
Model description

- agent: proximal policy optimization - stable baselines 3 (SB3)
- policy network: multilayer perceptron
- grid2op integration: PPO-SB3 from l2rpn-baselines wrapping SB3 agent
- SB3 incorporated several strategies for performance and stability on top of the original PPO paper

$$L^{CLIP}(\theta) = \hat{E}_t[min(r_t(\theta))\hat{A}_t, clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t)]$$

Environment

- L2RPN_case_14_sandbox split in train (80%), validation (20%) and test (10%) with periodic chronics shuffling (environment implementation)
- action space: agent uses only set_bus actions allowing the agent only topological actions on substations
- observation space: use baseline default features except redispatching and storage related ones
- heuristics: do-nothing until thermal limit and always reconnect are used through the baseline GymEnvWithRecoWithDN environment

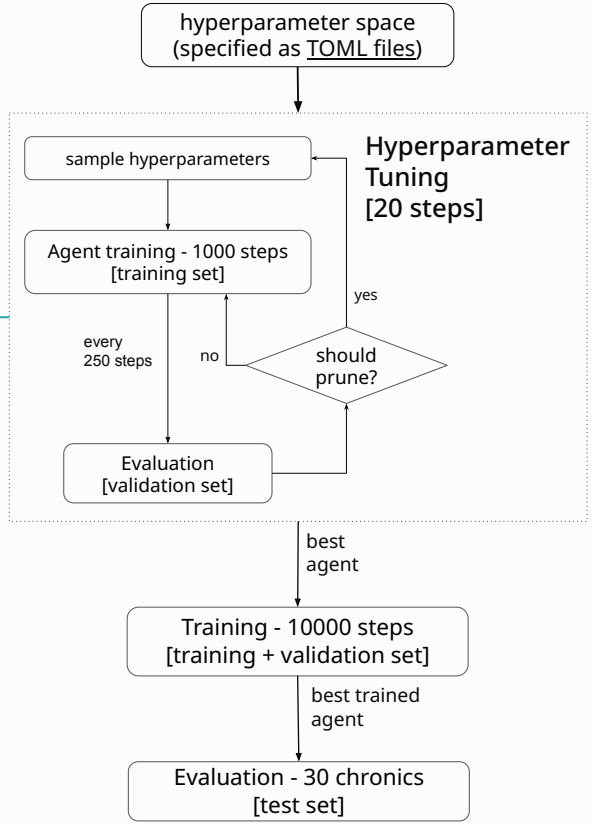


Training methodology

Hyperparameter tuning is implemented through Optuna with TPE sampling and Hyperband pruning. Pruning employs a custom callback (callback implementation) to periodically evaluate the model.
Tuning runs for 20 iterations for demonstration purposes due to computational constraints.

Reproducibility:

- all experiments run on google colab with notebooks
- all random generators are seeded
- evaluation and tensorboard data is made available
- tests ensure correctness before running expensive training jobs



Trained agent analysis

- Reward: EpisodeDurationReward
- Score: mean steps survived with respect to chronics length over 30 test chronics
- This reward has been determined as baseline as it is the nearest to the scoring function

No learning is happening:

- reward is too sparse, gets received only at the end of the episode makes learning very slow
- hyperparameters probably wrong, too few iterations (computational constraint)

batch size	gamma	learning rate	n_steps	net_arch	safe_max_rho
8	0.901	4e-5	128	200	0.95

Test score: 16%

BASELINE IMPROVEMENT

- Reward shaping
- Action masking
- Graph embeddings

Reward shaping

- Goal: stabilize learning with a more frequent reward that encodes how the network changes once an action has been played
- Solution: change environment reward class to LinesCapacityReward which is higher the more powerline capacity is available
- Conclusions:
  - learning is working correctly
  - thermal limit (safe max rho) is higher and may concur to the improved test score
- Future works: shaping through task specific penalties

Reward over 10k training steps

batch size	gamma	learning rate	n_steps	net_arch	safe_max_rho
32	0.96	9e-5	8	400	0.99

Test score: 29%

BASELINE IMPROVEMENT

Action Masking

- Goal: reduce the number of illegal actions that the agent is playing, the environment treats them as no action
- Solution: use MaskablePPO from sb3-contrib by subclassing baseline model (implementation) and subclassing base environment to provide action masks (implementation)
- Conclusions:
  - performance needs more iterations to be evaluated, same test score achieved also due to a small test set
  - safe_max_rho is lower, less no-action played by policy
- Future works: greedy masking through simulation

Reward over 10k training steps

batch size	gamma	learning rate	n_steps	net_arch	safe_max_rho
16	0.92	7e-5	64	400	0.97

Test score: 29%

BASELINE IMPROVEMENT

Graph embeddings

- Goal: leverage graph topology directly in the policy
- Solution: modify observation space adapter to flatten the observation energy graph (implementation) so that the agent can use a GNN policy (implementation) while the environment stays a vector environment
- Conclusions:
  - very noisy (almost absent) learning, GNN probably needs more regularization
- Future works: literature survey on effective architectures and strategies for stabilizing training

Reward over 10k training steps

Test score: 5%

(graph + maskable + improved reward)

batch size	gamma	learning rate	n_steps	safe_max_rho	gnn_dropout_p	gnn_features_dim	gnn_hidden_dim
16	0.97	3e-5	64	0.90	0.1	128	32

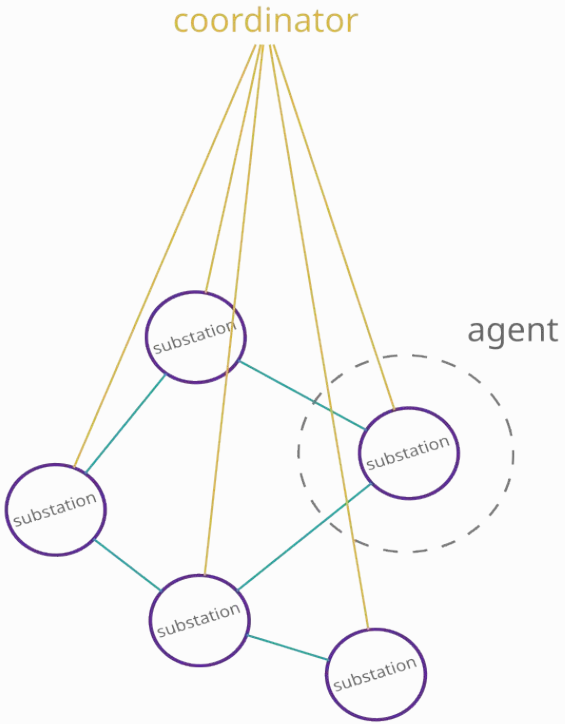
BASELINE IMPROVEMENT

MULTI-AGENT PPO

- High-level design and motivation
- Training and inference
- Implementation
- Challenges and mitigations

High-level design and motivation

- Reference paper: centrally coordinated multi-agent reinforcement learning for power grid topology control (arxiv)
- Solution description: one PPO agent per substation proposes an action together with its value (critic output) to a central coordinating agent that chooses what action to play then distributes the rewards
- L2RPN contribution: hierarchical MARL systems find a favourable tradeoff between scalability to real-world networks and stability.



Training and inference

Training:

1. if deterministic rules apply choose action through expert system
2. else each agent generates an action given its immediate substation neighbourhood
3. each agent sends to the coordinator the action and the value computed by its local critic
4. the coordinator takes the softmax of the action and uses the output as a weighting for sampling proposed actions
5. the sampled action gets played and each agent receives the reward, the agent whose action was chosen receives a bonus (configured as an hyperparameter). This is a modification to the paper in which only the chosen agent sees the reward and all other agents waste the sample. In an abstract sense each agent should receive the reward for communicating an action with the correct action-value even if it does not end up being chosen

Inference: actions do not get sampled but the coordinator chooses the action with the highest reward

Implementation

- the system must be wrapped in a single agent playing on the environment. This agent should be a Gym agent which stores all the required PPO agents in its state and the coordination logic. It is possible to reuse stable-baselines PPO implementations.
- the abstract agent should receive graph observations through a custom observation adapter (implementation) to be able to split the observation to each agent. Another possible way is to split the observation vector with a mask saved in the vector itself so to be able to split the observation for each substation. This just requires numpy and the compatibility layer built in grid2op
- action space must be also correctly decomposed for each substation so that each agent can only play local actions
- the agent itself would have a learn() method very similar to that of the standard PPO. This would be akin to the implementation of a IPPO system (example) where we iterate over the policies to update them according to the chosen action

Challenges and mitigations

- Greedy coordinator: using a simple sampling strategy may not be enough to model complex environments (e.g. attackers). In that case one could resort to implementing a RL coordinator that still sees proposed actions and action values but it also sees global observations of the environment. This would surely add expressivity but it would also add relevant system complexity.
- Credit assignment: using a reward which is controlled by a hyperparameter in the system is challenging as this value could be hard to select even with proper automatic tuning. Monitoring agent action selection and how the coordinator selects the agents together with the action value it receives is crucial in understanding if some agents get cut out from selection.
- Coordinator as a bottleneck: in this schema agents do not communicate among themselves but send and receive messages exclusively from the coordinator. Therefore the coordinator becomes a bottleneck whose failure would impede the functioning of the system at all. When reliability is critical the coordinator should be replicated among multiple machines and regions.